

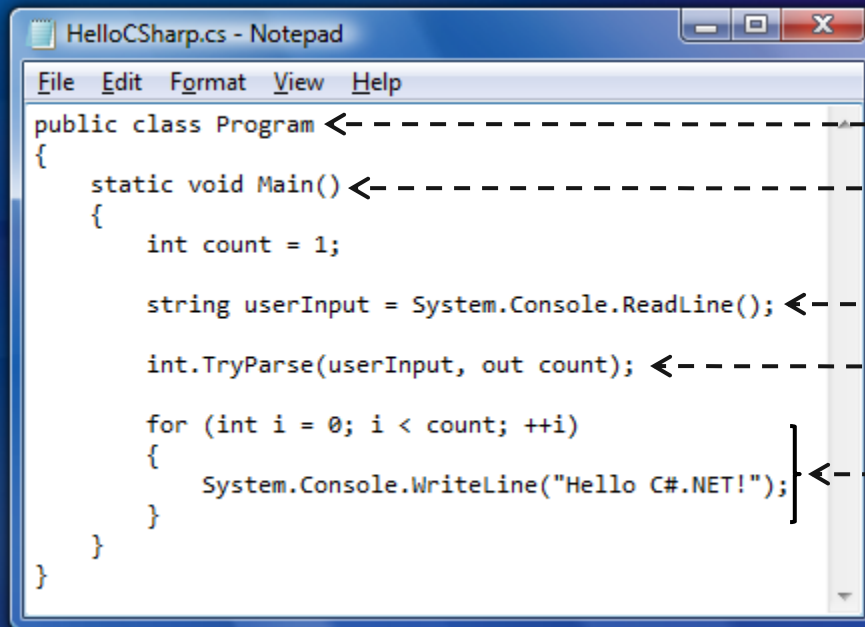
A large audience of people is seated in rows, viewed from the side. The image is overlaid with a blue gradient and a pattern of faint, rounded rectangular shapes. The text 'C#' is prominently displayed in the upper left corner.

C#

פרק 1

תחביר בסיסי

Hello C#.NET



```
File Edit Format View Help
public class Program
{
    static void Main()
    {
        int count = 1;

        string userInput = System.Console.ReadLine();

        int.TryParse(userInput, out count);

        for (int i = 0; i < count; ++i)
        {
            System.Console.WriteLine("Hello C#.NET!");
        }
    }
}
```

All code resides in classes

The entry point of the program

Use static methods of Console class for console I/O

Primitives are objects and have methods (i.e. count.ToString())

Syntax and statements are C/C++ like

אין קבצי ה.

Case Sensitive

מרחבי שמות - Namespaces

■ מאפשרים להגדיר טיפוסים באופן שלא ייצור כפילות שמות בעתיד

■ ניתן להגדיר מרחבי שמות מקוננים

■ ניתן לשוב והגדיר את אותו מרחב שמות גם בקבצים אחרים

■ ניתן לקצר את שמות הטיפוסים בקוד ע"י שימוש ב- `using`

```
1. using System.IO; // the 'using' statements are usually at the top of the file
2. namespace Chapter3.Namespaces
3. {
4.     using System; // a 'using' statement can be also inside a namespace
5.     namespace MyEnums
6.     (
7.         public enum eWeekDay
8.         {
9.             Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
10.        }
11.    System.Console.WriteLine(a); // using class Console without its namespace
12.    System.IO.File.Delete("foo.txt"); // using class File without its namespace
13.    }
14.    public class SomeClass
15.    {
16.        // eWeekDay is known by its short name when used in the same namespace.
17.        // (outside it is called Chapter3.Namespaces.MyEnums.eWeekDay
18.        MyEnums.eWeekDay m_Day = MyEnums.eWeekDay.Monday;
19.    }
20. }
```

מתודות ופרמטרים

לשעבר "פונקציות" ("פונקציה" תמיד מחזירה ערך. מתודות תמיד שייכות לטיפוס ולרוב הן- void)

ההכרזה והמימוש באותו מקום ותמיד בתוך טיפוס (שלא כמו ב-C, ++C)

חובה לאתחל משתנים מקומיים טרם השימוש בהם

ערכי הפרמטרים מועתקים בשליחה (כלומר נשלחים by value)

לא ניתן לציין ערכי ברירת מחדל לפרמטרים (רק החל מ-C# 4.0)

```
1. class Program {
2.     public static void Main()
3.     {
4.         int a; // an uninitialized local variable
5.         DoSomething(a); // error: can not use 'a' until it is set to a value
6.         a = 8; // a is initialized
7.         DoSomething(a);
8.         System.Console.WriteLine(a); // a's value is still 8.
9.         DoSomething(); // i_Num is defaulted to -1
10.    }

11.    public static void DoSomething(int i_Num = -1)
12.    {
13.        i_Num++; // i_Num is a by-value param, so this doesn't affect a's value
14.    }

15.    public static void DoSomething() // an overloading to the previous method
16.    {
17.        DoSomething(-1); // i_Num is defaulted to -1
18.    }
19. }
```

העברה ByValue או byRef?

```
// pass the int by value:
```

```
bool checkIfEven(int num)
{
    return num%2 == 0;
}
```

```
// in C language: pass a pointer to the int:
```

```
void swap(int* n1, int* n2)
{
    int temp = (*n1); (*n1) = (*n2); (*n2) = temp;
}
```

```
// in C++ language: pass a pointer to the int:
```

```
void swap(int & n1, int & n2)
{
    int temp = n1; n1 = n2; n2 = temp;
}
```

```
// C#:
```

```
void swap(ref int n1, ref int n2)
{
    int temp = n1; n1 = n2; n2 = temp;
}
```

מתודות ופרמטרים – המשך - ref

- נשתמש ב-ref כאשר יש משמעות לשינויים שהמתודה מבצעת לערך שנשלח, מחוץ למתודה, ובנוסף, יש משמעות לערך שנשלח גם בתוך המתודה.
- נכתוב ref גם בהכרזה של המתודה וגם בקריאה

```
1. class A
2. {
3.     public static void Main()
4.     {
5.         int a = 5;
6.         DoSomething(ref a);
7.         Console.WriteLine("a = {0}", a); // a = 6
8.     }
9.     public static void DoSomething(ref int io_X)
10.    {
11.        io_X++;
12.    }
13. }
```

■ חייבים לאתחל את המשתנה בערך לפני השליחה למתודה:

```
1.     public static void Main()
2.     {
3.         int a; // a was not initialized
4.         DoSomething(ref a); // ERROR!! (a must be initialized before the send)
5.         Console.WriteLine("a = {0}", a); //ERROR!!
6.     }
```

מתודות ופרמטרים – המשך - out

לצורך שליחה למתודה "תיבה ריקה" להחזרת ערכים
במתודה, הפרמטר נחשב לא מאותחל (ולכן חובה לאתחל במתודה לפני השימוש)
יש משמעות להשמה שהמתודה מבצעת לערך שנשלח, מחוץ למתודה
יש צורך להכריז כ- out גם בהכרזה וגם בקריאה

```
1. class A
2. {
3.     public static void Main()
4.     {
5.         int a;
6.         DoSomething(out a); // a was not initialized but it's ok
7.         Console.WriteLine("a = {0}", a); // a = 1;
8.     }
9.     public static void DoSomething(out int o_X)
10.    {
11.        o_X = 0;
12.        o_X++;
13.    }
14. }
```

אין חובה לאתחל לפני השליחה
חובה לאתחל בגוף המתודה לפני השימוש

```
1. public static void DoSomething(out int o_X)
2. {
3.     o_X++; // error!! 'o_X' is used before initialization
4. }
```

מתודות ופרמטרים – המשך - params

Params (דומה ל ... "שלוש נקודות" ב-Java ו-C++)

- ניתן להוסיף לפני פרמטר מסוג מערך, כדי להקל על השולח ולאפשר לו לשלוח רשימת פרמטרים מופרדים בפסיק, במקום לבנות בעצמו מערך
- אפשרי רק בפרמטר האחרון

```
1. class A
2. {
3.     public static void Main()
4.     {
5.         int result;
6.         result = DoSomething(7, 1, 2, 3);
7.         Console.WriteLine("result = {0}", result);
8.     }
9.
10.    public static int DoSomething(int i_Times, params int[] i_Arr)
11.    {
12.        int sum = 0;
13.        for(int i = 0 ; i < i_Arr.Length ; ++i);
14.        {
15.            sum += i_Arr[i];
16.        }
17.        return sum * i_Times;
18.    }
```

if/while - אין המרה אוטומטית ל-bool

```

1.  int numOfStudents = getNumOfStudents();
2.  while (numOfStudents) // Error: no implicit casting from int to bool
3.  {
4.      // ...
5.  }
6.  if ((bool)numOfStudents) // Error: no casting from int to bool
7.  {
8.      // ...
9.  }
10. if (numOfStudents = 0) // Error: no implicit casting from int to bool
11. {
12.     // ...
13. }

14. Item head = getHeadOfList();
15. if (head) // Error: no implicit casting from Item to bool
16. {
17.     // ...
18. }
19. while ((bool)head.Next) // Error: no casting from Item to bool
20. {
21.     // ...
22. }
23. if (head.Next = null) // Error: no implicit casting from Item to bool
24. {
25.     // ...
26. }
    
```

ביטויים - המשך

switch

עובד גם על מחרוזות (לא מומלץ)

אין אפשרות ל"גלישה אוטומטית" בין case אחד לשני

Must put **break**, **goto**, **return**, **throw**, **continue** (if inside a loop)

for

המשתנה שרץ לא מוכר מחוץ ללולאה

break – עוצר לולאה

כמו ב- C++ ו- Java

continue – ממשיך לאיטרציה הבאה בלולאה

כמו ב- C++ ו- Java